Tarea 5: Simulador de RISC-V

> "The most effective debugging tool is still careful thought, coupled with judiciously placed print statements."
>
> — Brian Kerninghan

Se solicita hacer un simulador del ISA de RISC-V visto en clase. Este simulador debe permitir cargar un programa compilado desde un archivo y poder ejecutar el código de máquina paso por paso, permitiendo inspección de registros y memoria.

A nivel conceptual, el simulador debe mantener el estado arquitectural de una CPU RISC-V: contador de programa (PC), 32 registros y memoria. La ejecución de una instrucción consiste en leer la instrucción apuntada por el PC, decodificarla, ejecutar su comportamiento y actualizar el estado del simulador.

Se puede tomar como ejemplo de un simulador en C++ para un ISA ficticio el siguiente repositorio: `https://github.com/Grazen0/toy-simulator`. **(Revisar `src/main.cpp`)**. Se puede tomar como guía para comprender las nociones básicas de cómo simular un ISA.

1. Requisitos

1.1. Obligatorios

- Soporte para el ISA base RISC-V de 32 bits (ver anexo A).
  - La memoria del simulador debe seguir el formato little-endian de RISC-V.
- Permitir la carga de programas (**ya compilados**) desde archivos.
  - El programa se debe cargar al address 0x00000000.
  - El archivo de programa debe ser un binario crudo. Ver anexo B para más información.
- Soportar ejecución paso por paso. (Es decir, ejecutar instrucciones una por una de forma controlada por el usuario).
- Permitir inspección de PC, registros y memoria, de tal forma que se puedan ver sus valores en distintos puntos de ejecución de un programa.

1.2. Ideas adicionales

- Soporte para `ecall` con algunas de las environment calls de SPIM especificadas en `https://cpulator.01xz.net/doc/#syscall`. Entre otras cosas, esto se puede usar para que un programa pueda imprimir texto y tomar input.

- Soporte para cargar programas de archivos `.elf` directamente.
- Incluir un desensamblador sencillo que permita ver las instrucciones originales del programa en ejecución.
- Soporte para uso con GDB.

2. Consideraciones

No se pide hacer un ensamblador de RISC-V, sino un simulador que ejecute código de máquina RISC-V ya compilado. El anexo B contiene instrucciones para descargar un programa compilado de CPUlator.

El lenguaje de implementación es de elección libre. Por conveniencia de desarrollo, se recomienda altamente el uso de un lenguaje de sistemas (C, C++, Rust, etc.). Sin embargo, un lenguaje como JavaScript en el navegador puede facilitar la integración de una interfaz gráfica.

No es necesario modelar permisos o segmentación de memoria. Basta con una memoria plana byte-addressable con un address space 32 bits. La implementación concreta de dicha memoria queda a elección libre.

El comportamiento ante instrucciones inválidas queda a criterio de la implementación. Se recomienda reportar errores de ejecución de forma explícita.

Además, se recomienda probar el simulador con al menos los siguientes programas:

- **Quicksort** (`https://pastebin.com/MmQzpFyR`): Coloca el array {`6, 4, 3, 2, 1, 8, 9`} en la dirección 0x1000, aplica quicksort in-place recursivamente al arreglo y lo ordena, dejando {`1, 2, 3, 4, 6, 8, 9`} en esa misma dirección.
- **Árbol simétrico** (`https://pastebin.com/R74Uqc68`): Coloca el árbol completo aplanado {`6, 4, 4, 1, 2, 2, 1`} en la dirección 0x1000 y evalúa si es un árbol simétrico con la función `is_symmetric`. Por serlo, el programa culmina con `a0 = 1` (true).
- **riscvtest.s** (`https://pastebin.com/pKuqWTLh`): Prueba las instrucciones `add`, `sub`, `and`, `or`, `slt`, `addi`, `lw`, `sw`, `beq` y `jal`. Si todo sale bien, escribe el número 25 (0x19) a la dirección 100 (0x64) en memoria.

3. Entregables

En canvas se deberá enviar un documento .pdf con el siguiente contenido:

- Enlace al repositorio Git del simulador.
- Enlace a una carpeta en Google Drive con un video demostrativo del simulador y el código fuente (como ZIP o carpeta). El video debe incluir una breve explicación del simulador y mostrar por lo menos 4 ejemplos de ejecución.
- Informe breve del simulador implementado (lenguaje, features, etc.).

4. Inspiración

4.1. Para terminal

```
$ ./build/awesome-simulator programa.bin
"programa.bin" cargado a memoria.
> pc
pc = 0x00000000
> step
Ejecutando instrucción.
> step
Ejecutando instrucción.
> step
Ejecutando instrucción.
> pc
pc = 0x0000000C
> regs x5 x14
x5 = 0x00000123
x14 = 0x10200405
> step
Ejecutando instrucción.
> mem 0x1000 0x1003
Memoria (0x1000-0x1003): 00 05 7C E8
> exit
See you next time...
Program exited with code 0.
```

4.2. Para navegador

Se puede tomar como inspiración CPUlator (`https://cpulator.01xz.net/?sys=rv32`, ver fig. 1), aunque solo se piden ciertas funcionalidades esenciales (ejecutar un programa, inspeccionar registros y memoria, etc.).

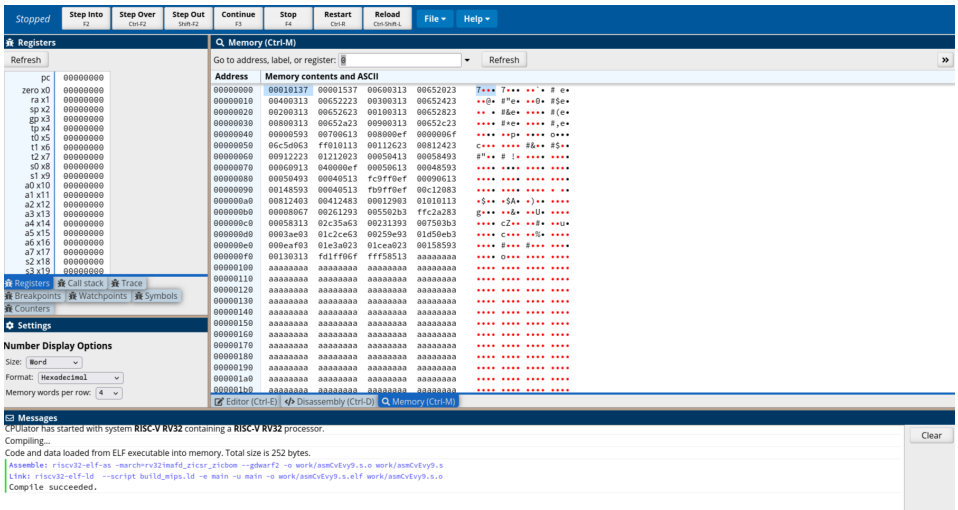


Figura 1: Ejemplo de un simulador de RISC-V en el navegador.

A. Instrucciones a implementar

Instrucción	**Descripción**
`lb rd, imm(rs1)`	Load byte
`lh rd, imm(rs1)`	Load halfword
`lw rd, imm(rs1)`	Load word
`lbu rd, imm(rs1)`	Load byte unsigned
`lhu rd, imm(rs1)`	Load halfword unsigned
`addi rd, rs1, imm`	Add immediate
`slli rd, rs1, shamt`	Shift left logical immediate
`slti rd, rs1, imm`	Set less than immediate
`sltiu rd, rs1, imm`	Set less than immediate unsigned
`xori rd, rs1, imm`	XOR immediate
`srli rd, rs1, shamt`	Shift right logical immediate
`srai rd, rs1, shamt`	Shift right arithmetic immediate
`ori rd, rs1, imm`	OR immediate
`andi rd, rs1, imm`	AND immediate
`auipc rd, imm`	Add upper immediate to PC
`sb rs2, imm(rs1)`	Store byte
`sh rs2, imm(rs1)`	Store halfword
`sw rs2, imm(rs1)`	Store word
`add rd, rs1, rs2`	Add
`sub rd, rs1, rs2`	Subtract
`sll rd, rs1, rs2`	Shift left logical
`slt rd, rs1, rs2`	Set less than
`sltu rd, rs1, rs2`	Set less than unsigned
`xor rd, rs1, rs2`	XOR
`srl rd, rs1, rs2`	Shift right logical
`sra rd, rs1, rs2`	Shift right arithmetic
`or rd, rs1, rs2`	OR
`and rd, rs1, rs2`	AND
`lui rd, imm`	Load upper immediate
`beq rs1, rs2, offset`	Branch if equal
`bne rs1, rs2, offset`	Branch if not equal
`blt rs1, rs2, offset`	Branch if less than
`bge rs1, rs2, offset`	Branch if greater than or equal
`bltu rs1, rs2, offset`	Branch if less than unsigned
`bgeu rs1, rs2, offset`	Branch if greater than or equal unsigned
`jalr rd, rs1, imm`	Jump and link register
`jal rd, offset`	Jump and link

Cuadro 1: Instrucciones a implementar en el simulador.

B. Cómo descargar un programa compilado en CPUlator

Para obtener un programa compilado en el formato binario que tu simulador debe leer, puedes seguir los siguientes pasos en CPUlator:

1. Habiendo hecho tu programa, hacer click en **“Compile and Load”** (F5).
2. Ir a la pestaña **“Memory”** (Ctrl+M).
3. Expandir el menú lateral escondido a la derecha (ver fig. 2).
4. Bajo “File format”, **establecer “Format” como “Raw”**. (Ver fig. 3.)
5. Llenar los siguientes campos bajo “Save” (ver fig. 3):
   - Start address: `0000`
   - End address: *(Alguna dirección tal que el rango especificado contenga todo el programa.)*
6. Hacer click en **“Save to file”**.

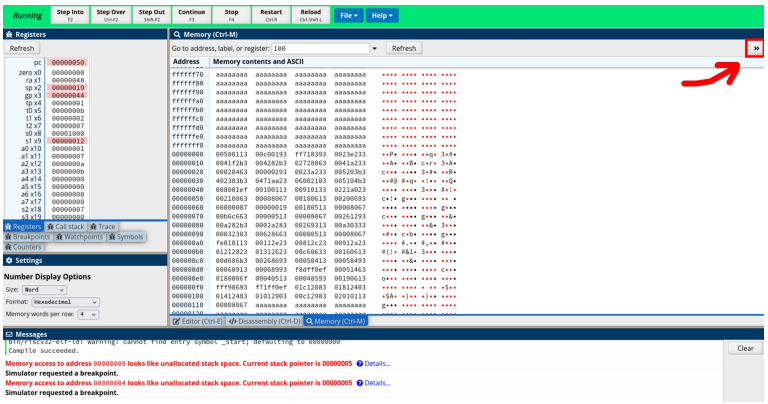


Figura 2: Botón para expandir el menú lateral derecho en CPUlator.

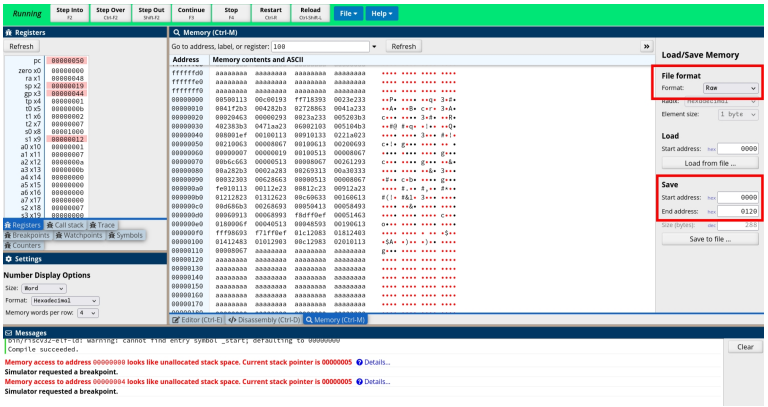


Figura 3: Menú de “Load/Save Memory” de CPUlator con campos correctamente establecidos.

Otra opción es hacer click en “File” → “Save...” (Ctrl+S) en la barra superior, pero esto te dará un archivo ejecutable de Linux (`.elf`), no un binario con el programa compilado en crudo. *(Pero podrías implementar soporte para archivos ELF...)*